

JAVA OOP PRACTICAL RECORD

Digital Wallet – UPI Payment System

Experiment No. 05

Title: Digital Wallet – Exception Handling and Interface

1. Objective / Aim

To write a Java program that demonstrates exception handling and interface concepts through a Digital Wallet and UPI Payment System, including custom exceptions, encapsulation, interface implementation, validation, try-catch-finally, and menu-driven operations.

2. System Requirements

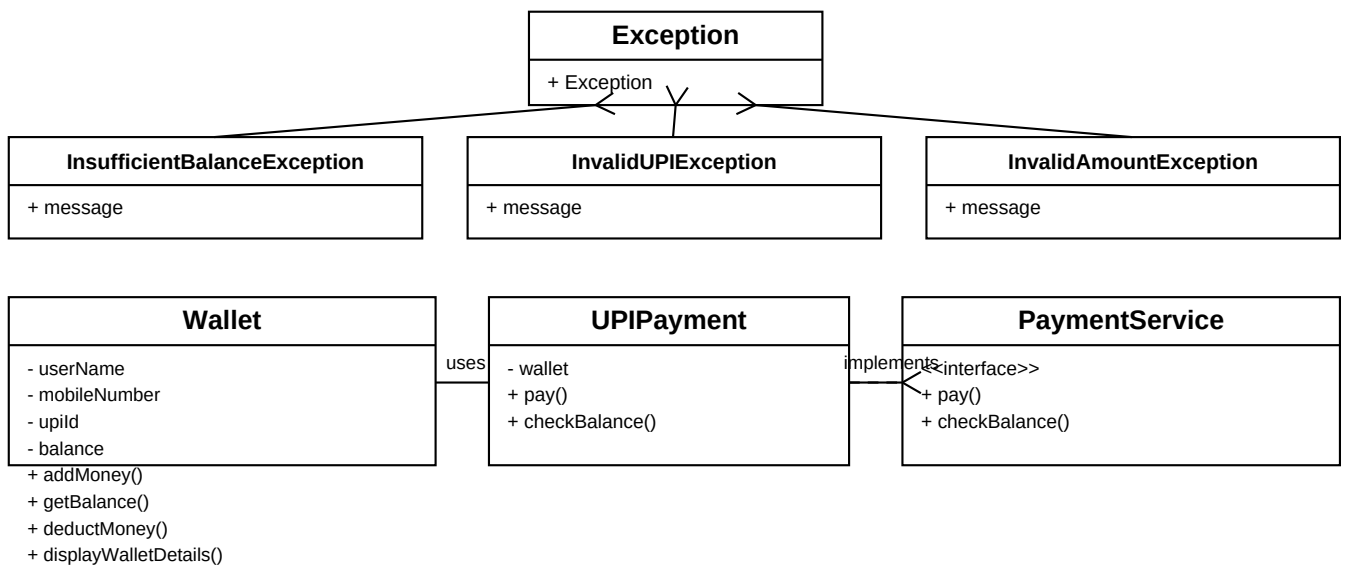
Hardware: Standard PC/Laptop with 4 GB/8 GB RAM and a Core i3/i5 processor or equivalent.

Software: Windows/Linux/macOS, JDK 17 or JDK 21, and a Java editor or IDE such as VS Code, Eclipse, IntelliJ IDEA, or NetBeans.

3. Algorithm / Logic Description

1. Create custom exception classes `InsufficientBalanceException`, `InvalidUPIException`, and `InvalidAmountException`.
2. Create the `PaymentService` interface with `pay()` and `checkBalance()` methods.
3. Create the `Wallet` class with private data members for user details, UPI ID, and balance.
4. Implement `addMoney()`, `getBalance()`, `deductMoney()`, and `displayWalletDetails()` in `Wallet`.
5. Create `UPIPayment` implementing `PaymentService` and validate the receiver UPI ID, payment amount, and wallet balance.
6. Throw the appropriate custom exception when validation fails.
7. Use try-catch blocks to handle transaction errors and finally to indicate transaction processing completion.
8. In `main()`, accept wallet details using `Scanner` and provide a menu for adding money, making UPI payments, checking balance, displaying wallet details, and exiting.

4. UML Class Diagram



5. Source Code

```
package observation.exp05;
import java.util.Scanner;

// ===== CUSTOM EXCEPTIONS =====
class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}
class InvalidUPIException extends Exception {
    public InvalidUPIException(String message) {
        super(message);
    }
}
class InvalidAmountException extends Exception {
    public InvalidAmountException(String message) {
        super(message);
    }
}

// ===== INTERFACE =====
interface PaymentService {
    void pay(String upiId, double amount)
        throws InvalidUPIException, InvalidAmountException,
        InsufficientBalanceException;
    void checkBalance();
}

// ===== WALLET CLASS =====
class Wallet {
    private String userName;
    private String mobileNumber;
    private String upiId;
    private double balance;

    public Wallet(String userName, String mobileNumber,
        String upiId, double balance) {
        this.userName = userName;
        this.mobileNumber = mobileNumber;
        this.upiId = upiId;
        this.balance = balance;
    }

    public void addMoney(double amount)
        throws InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException(
                "Amount must be greater than zero.");
        }
        balance = balance + amount;
        System.out.println(
            "■" + amount + " added successfully.");
    }

    public double getBalance() {
```

5. Source Code (continued)

```
        return balance;
    }

    public void deductMoney(double amount) {
        balance = balance - amount;
    }

    public void displayWalletDetails() {
        System.out.println("\n===== WALLET DETAILS =====");
        System.out.println("User Name      : " + userName);
        System.out.println("Mobile Number : " + mobileNumber);
        System.out.println("UPI ID       : " + upiId);
        System.out.println("Balance      : ■" + balance);
    }
}

// ===== UPI PAYMENT CLASS =====
class UPIPayment implements PaymentService {
    private Wallet wallet;

    public UPIPayment(Wallet wallet) {
        this.wallet = wallet;
    }

    public void pay(String upiId, double amount)
        throws InvalidUPIException,
               InvalidAmountException,
               InsufficientBalanceException {

        if (upiId == null ||
            !upiId.contains("@") ||
            upiId.startsWith("@") ||
            upiId.endsWith("@")) {
            throw new InvalidUPIException(
                "Invalid UPI ID.");
        }

        if (amount <= 0) {
            throw new InvalidAmountException(
                "Payment amount must be greater than zero.");
        }

        if (amount > wallet.getBalance()) {
            throw new InsufficientBalanceException(
                "Insufficient wallet balance.");
        }

        wallet.deductMoney(amount);

        System.out.println("\n===== PAYMENT SUCCESSFUL =====");
        System.out.println("Paid To : " + upiId);
        System.out.println("Amount  : ■" + amount);
    }

    public void checkBalance() {
```

5. Source Code (continued)

```
        System.out.println(
            "Available Balance: ■" + wallet.getBalance());
    }
}

// ===== MAIN CLASS =====
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("==== CREATE WALLET =====");

        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        System.out.print("Enter mobile number: ");
        String mobile = sc.nextLine();

        System.out.print("Enter your UPI ID: ");
        String myUpi = sc.nextLine();

        System.out.print("Enter initial balance: ");
        double balance = sc.nextDouble();

        Wallet wallet = new Wallet(
            name, mobile, myUpi, balance);

        UPIPayment payment = new UPIPayment(wallet);
        int choice;

        while (true) {
            System.out.println("\n=====");
            System.out.println("        DIGITAL WALLET");
            System.out.println("=====");
            System.out.println("1. Add Money");
            System.out.println("2. Make UPI Payment");
            System.out.println("3. Check Balance");
            System.out.println("4. Display Wallet Details");
            System.out.println("5. Exit");
            System.out.println("=====");

            System.out.print("Enter your choice: ");
            choice = sc.nextInt();

            switch (choice) {
                case 1:
                    System.out.print(
                        "Enter amount to add: ");
                    double addAmount = sc.nextDouble();

                    try {
                        wallet.addMoney(addAmount);
                    }
                    catch (InvalidAmountException e) {
                        System.out.println(
```

5. Source Code (continued)

```
                "Transaction Failed: " +
                e.getMessage());
        }
        break;

    case 2:
        sc.nextLine();

        System.out.print(
            "Enter receiver UPI ID: ");
        String receiverUPI = sc.nextLine();

        System.out.print(
            "Enter payment amount: ");
        double paymentAmount = sc.nextDouble();

        try {
            payment.pay(
                receiverUPI, paymentAmount);
        }
        catch (InvalidUPIException e) {
            System.out.println(
                "Transaction Failed: " +
                e.getMessage());
        }
        catch (InvalidAmountException e) {
            System.out.println(
                "Transaction Failed: " +
                e.getMessage());
        }
        catch (InsufficientBalanceException e) {
            System.out.println(
                "Transaction Failed: " +
                e.getMessage());
        }
        finally {
            System.out.println(
                "Transaction processing completed.");
        }
        break;

    case 3:
        payment.checkBalance();
        break;

    case 4:
        wallet.displayWalletDetails();
        break;

    case 5:
        System.out.println(
            "\nThank you for using Digital Wallet!");
        sc.close();
        return;
```

5. Source Code (continued)

```
        default:
            System.out.println(
                "Invalid choice! Please enter 1 to 5.");
    }
}
}
```

6. Compilation & Execution Commands

Because the program contains a package declaration, save the file as **Main.java** inside the folder structure **observation/exp05**.

```
javac -d . Main.java
java observation.exp05.Main
```

7. Sample Output Screen

```
===== CREATE WALLET =====
Enter your name: Avinash
Enter mobile number: 9876543210
Enter your UPI ID: avinash@upi
Enter initial balance: 1000

=====
                DIGITAL WALLET
=====
1. Add Money
2. Make UPI Payment
3. Check Balance
4. Display Wallet Details
5. Exit
=====
Enter your choice: 2
Enter receiver UPI ID: shop@upi
Enter payment amount: 250

===== PAYMENT SUCCESSFUL =====
Paid To : shop@upi
Amount : ■250.0
Transaction processing completed.

Enter your choice: 3
Available Balance: ■750.0

Enter your choice: 5
Thank you for using Digital Wallet!
```

8. Result

The Digital Wallet – UPI Payment System was successfully implemented. The program demonstrates custom exception handling for invalid amounts, invalid UPI IDs, and insufficient balance, along with interface implementation, encapsulation, validation, try-catch-finally, and menu-driven operations.